IRL-DAL: Safe and Adaptive Trajectory Planning for Autonomous Driving via Energy-Guided Diffusion Models

Seyed Ahmad Hosseini Miangoleh [1], Amin Jalal Aghdasian [1], Farzaneh Abdollahi [1]
[1]Department of Electrical Engineering, Amirkabir University of Technology (Tehran Polytechnic), Tehran, Iran

***Abstract*—This paper proposes a novel inverse reinforcement learning framework using a diffusion-based adaptive lookahead planner (IRL-DAL) for autonomous vehicles. Training begins with imitation from an expert finite state machine (FSM) controller to provide a stable initialization. Environment terms are combined with an IRL discriminator signal to align with expert goals. Reinforcement learning (RL) is then performed with a hybrid reward that combines diffuse environmental feedback and targeted IRL rewards. A conditional diffusion model, which acts as a safety supervisor, plans safe paths. It stays in its lane, avoids obstacles, and moves smoothly. Then, a learnable adaptive mask (LAM) improves perception. It shifts visual attention based on vehicle speed and nearby hazards. After FSM-based imitation, the policy is fine-tuned with Proximal Policy Optimization (PPO). Training is run in the Webots simulator with a two-stage curriculum. A 96% success rate is reached, and collisions are reduced to 0.05 per 1k steps, marking a new benchmark for safe navigation. By applying the proposed approach, the agent not only drives in lane but also handles unsafe conditions at an expert level, increasing robustness.We make our code publicly available[1]**

***Index Terms*—Autonomous Vehicles, Diffusion Models, Trajectory Planning, Inverse Reinforcement Learning (IRL), Adaptive Attention, Proximal Policy Optimization (PPO), Hybrid Reward**

I. Introduction

The main challenge in autonomous vehicles is building systems that can operate at human levels of safety and reliability in highly dynamic environments [1], [2]. This challenge mainly comes from the need to avoid obstacles in a strong and reliable way. Obstacle avoidance is the key safety task that shows whether a system can work in the real world. Even small mistakes in rare situations can cause very serious failures. This means the agent must remain safe even when encountering situations it was not trained on [3]. To address these challenges, recent research has investigated a range of advanced methods. These approaches aim to enhance the safety, reliability, and adaptability of autonomous systems operating in dynamic environments.

A. Related Work

This work falls within the overlap of four main areas: hybrid learning, reward inference, generative planning, and adaptive perception. The following section reviews some recent studies in each of these areas.

[1]Paper Website

1) Hybrid Imitation and Reinforcement Learning: Imitation Learning (IL), especially Behavioral Cloning (BC), is widely used in autonomous driving [4]. It provides a data-efficient way to learn a mapping from expert demonstrations to control actions using supervised learning [5], [6]. The primary advantage of BC is its computational efficiency and its requirement for no explicit knowledge of the underlying environment dynamics. Within BC, methods such as Conditional Imitation Learning improve performance by conditioning the policy on high-level commands. This helps address the challenge of having many possible correct actions for a given situation [7]. Even though it is efficient, IL has a problem called covariate shift. In this case, small mistakes grow over time when the agent reaches states that were not in the training data. Unlike traditional methods, RL can develop strong and resilient behaviors by learning from trial-and-error experience. However, it usually needs a lot of data and depends on hand-designed reward signals that can be unreliable [8].

The hybrid method combines IL and RL. This lets the self-driving car first learn basic behavior from examples and then improve it through trial, feedback, and optimization [9]. In [10], a hybrid method is used that combines BC with the PPO algorithm in the Unity Agents environment. This setup trains racecar agents to drive and avoid obstacles. These combinations capitalize on expert supervision for fast convergence while allowing the agent to explore recovery and adaptation strategies [11]. In [12], the BC-SAC model fuses BC and Soft Actor-Critic to enhance policy robustness and safety in autonomous driving. By employing supervised imitation and reinforcement optimization, one achieves higher generalization and a 38% reduction in failure rate across complex real-world scenarios.

2) Inverse Reinforcement Learning for Reward Shaping: The problem of designing a good reward for complex tasks can be handled with Inverse Reinforcement Learning (IRL). IRL tries to find the hidden reward rules directly from expert demonstrations [13], [14]. The Conditional Predictive Behavior Planning model combines Conditional Motion Prediction and Maximum Entropy IRL to make driving more like a human. It predicts how nearby cars will react to each possible move and scores these moves using expert driving data [15]. Adversarial inverse reinforcement learning (AIRL) uses a GAN to learn the reward and the driving policy at the same time, which helps the agent adapt to new environments. Safety-aware AIRL then adds safety rules to block risky actions and lower the chance of crashes [16].

3) Adaptive Perception and Attention in Driving: With the growing use of attention mechanisms in deep learning [17], [18], these methods have been added to end-to-end driving models. They help the system focus more on key visual elements such as vehicles, pedestrians, and road signs [19]. Effective driving requires perception systems that can adapt attention dynamically to changing contexts. In [20],[21], it uses spatiotemporal, uncertainty-aware attention over multimodal sensors with crossmodal alignment and multiscale fusion to prioritize important actors and regions for downstream planning. In [22], it adds a temporal residual block, multiscale feature fusion, and global plus double attention to use time and image cues better. In [23], it combines adaptive channel attention and grouped spatial attention with channel shuffle to highlight important features. It plugs into standard CNNs, can replace a 3×3 convolution layer, and improves accuracy with little extra compute.

4) Diffusion Models for Safe Motion Planning: Diffusion models [24] are now leading tools for generating data. They are also being used more and more for planning tasks. When generating trajectories, they can produce diverse future motions that still obey real-world physics [25]. Their key advantage is their flexibility. By guiding the backward diffusion steps with extra rules or helper models, they can add safety rules such as avoiding crashes and keeping motions that the vehicle can really follow [26], [27]. Current methods use diffusion models as direct policy models, mainly in offline reinforcement learning settings [28]. Moreover, some methods use stand-alone planners that first produce open-loop trajectories. A separate controller then follows these trajectories .

B. Research Gap

Despite progress in each area, a key gap appears at their intersection. Current stacks rarely combine generative planning, reward inference, and online policy learning into one system. This causes three main limits:

1) Lack of an end-to-end unified loop: Most diffusion-based planners generate trajectories open loop and are executed separately from the RL policy, creating a distribution mismatch between planned motions and closed-loop control. This separation weakens robustness under disturbances and hinders consistent transfer from trajectory proposals to joint steering–speed commands.
2) Non-adaptive safety trade-offs: Fixed cost weights for lane keeping, collision avoidance, and stability cannot re-balance as scene risk and sensor uncertainty change. Without real-time, perception-conditioned guidance, systems oscillate between over-conservative and overly aggressive behavior and fail to maintain safety while preserving efficiency.
3) Inefficient learning signals: Reliance on sparse, hand-crafted rewards leads to sample-hungry training, unstable convergence, and sub-expert driving. The absence of dense rewards inferred from demonstrations and a staged curriculum limits generalization to out-of-distribution states and degrades control smoothness.

C. Motivation

Reliable autonomous driving must jointly address three coupled challenges: learning efficiency, decision safety, and perceptual adaptability.

Learning Pure RL needs a lot of data and becomes unstable when reward signals are rare. IL uses data well but still drifts under covariate shift, and detailed rewards made by people often fail to match expert goals in complex traffic.

Safety and Planning Reactive policies are fast to respond, but cannot plan very far ahead. Generative planners can guess future events, but they often cannot run in real time or give strong safety guarantees, so there is still a gap between long-term planning and moment-to-moment control.

Perception Standard vision encoders treat almost all regions of an image uniformly and struggle to leverage contextual information effectively. In driving, however, attention mechanisms should be dynamic: prioritizing near-field lane coherence to ensure lateral stability at high speeds, while intensifying focus on immediate proximity when potential hazards are detected.

These limitations highlight the need for a unified framework that connects stable imitation with exploratory reinforcement learning.

D. Contributions

We introduce a IRL–DAL, a cohesive framework that confronts the above challenges through three complementary components:

(1) Hybrid IL–IRL–RL Training A structured pipeline that integrates BC for initialization with PPO fine-tuning under a hybrid reward combining sparse environment feedback and dense GAIL-based intent rewards. This ensures stability, sample efficiency, and policy alignment with expert intent.

(2) Diffusion Planner for On-Demand Safety A conditional diffusion model serves as a short-horizon, risk-aware planner, activated only in uncertain or high-risk states. It generates candidate trajectories optimized by an energy-based objective penalizing collisions and abrupt control variations, allowing the main policy to internalize safer behaviors via planner feedback.

(3) Learnable Adaptive Mask (LAM) A lightweight perception module that dynamically modulates spatial attention based on vehicle kinematics and LiDAR proximity. The mask directs the visual encoder toward context-critical regions—amplifying lower-field road features at high speed for precise lane keeping and highlighting proximate surroundings near hazards—achieving interpretable and efficient attention allocation without heavy self-attention overhead.

II. Problem Formulation

The paper first formalizes the autonomous driving task as a partially observable Markov decision process POMDP defined by the tuple:

$$(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \mathcal{O}, \mathcal{Z}, \gamma)$$

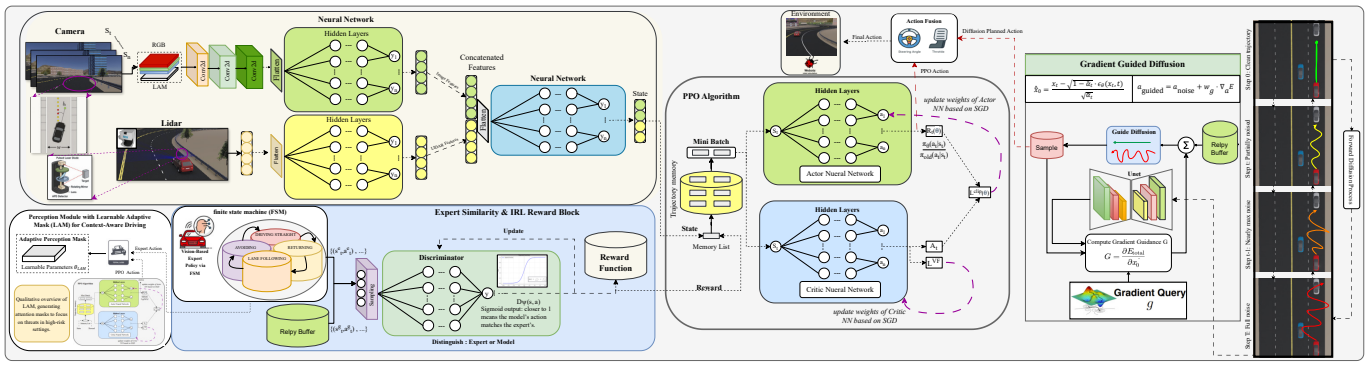

Fig. 1. Overview of the IRL-DAL architecture. The training process unfolds in two phases: the policy is first initialized by Behavioral Cloning (BC) and then fine-tuned with Proximal Policy Optimization (PPO) using a hybrid reward r_{total}. A LAM enhances state-aware perception. During RL rollouts, the Diffusion-based Adaptive Lookahead (DAL) planner serves as a safety supervisor, correcting unsafe PPO actions through energy-guided sampling so that only safe experiences are stored in the replay buffer $\mathcal{D}_{\text{PPO}}$.

which captures the agent partial observability and reliance on noisy sensor data.

State Space ($\mathcal{S}$) The latent true state includes the ego car position, orientation, and speed. It also includes nearby moving objects, the road layout. The policy cannot see this state directly.

Action Space ($\mathcal{A}$) Each action is a continuous two-dimensional vector:

$$a_t = [\,\text{steering},\ \text{speed}\,]$$

which is later mapped by a low-level controller into executable control commands. This design provides smooth, normalized continuous control.

Transition Function ($\mathcal{T}$) The stochastic dynamics:

$$\mathcal{T}(s_{t+1} \mid s_t, a_t)$$

defines how the true state evolves given the action.

Observation Space ($\mathcal{O}$) The multimodal observation at each time step is given by:

$$o_t = \{I_t, L_t, K_t\}$$

Where:

- **Camera Image** ($I_t \in \mathbb{R}^{H\times W\times 3}$): front-facing RGB frame.
- **LiDAR Scan** ($L_t \in \mathbb{R}^{180}$): range readings from a forward LiDAR.
- **Vehicle Kinematics** (K_t): normalized ego-vehicle speed.

The LAM is generated internally based on K_t and L_t. Thus, they belong to the perception module rather than the raw observation space.

Observation Function ($\mathcal{Z}$) The conditional distribution:

$$\mathcal{Z}(o_t \mid s_{t+1}, a_t)$$

Models the sensing process that generates observations.

Reward Function ($\mathcal{R}$) Since the agent perceives the environment only through observations, the reward depends on both o_t and a_t. IRL-DAL uses a hybrid reward. It combines an environment-defined term with an intrinsic term learned by the IRL discriminator.

$$r_t(o_t, a_t) = (1 - w_{\text{IRL}})\, r_{\text{env}}(o_t, a_t) + w_{\text{IRL}}\, r_{\text{IRL}}(o_t, a_t)$$

where w_{IRL} is a phase-dependent weight used during the mixed training phase, the influence of the learned reward throughout training. Both terms depend solely on observable quantities, ensuring consistency under partial observability.

Discount Factor (γ) A scalar γ that balances short-term rewards with long-term performance.

Objective The agent aims to learn a stochastic policy $\pi_\theta(a_t \mid o_t)$, parameterized by θ, that maximizes the expected discounted return:

$$\mathbb{E}_{\pi_\theta,\,\mathcal{T},\,\mathcal{Z}}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$$

III. Methodology

Figure 1 provides an overview of the entire architecture. Our proposed framework integrates safety, stability, and expert-like decision-making within a single autonomous driving system. As illustrated in Figure 1, the architecture combines four interacting components that collectively enable robust and adaptive behavior: (1) context-aware perception via the LAM, (2) FSM-aware structured replay buffers, (3) PPO fine-tuning with a hybrid reward, and (4) diffusion-based safety supervision with experience correction.

A. Perception Module

The perception module uses a learnable attention mechanism that dynamically focuses on the driving situation. This adaptive mask helps the agent build a compact but informative state representation s_t from high-dimensional, multi-sensor inputs. The quality of the driving policy strongly depends on how well it sees and encodes the environment. Standard end-to-end methods often treat all image pixels equally, wasting model capacity. To fix this, we introduce the LAM, a small differentiable module that adds safety-related prior knowledge to the visual input using top-down attention.

Figure 2 shows the LAM architecture and how it connects to the PPO policy. The module takes two inputs: the current vehicle speed v_t and the minimum LiDAR distance $d_{\min,t} = \min(\min(l_t), d_{\max})$, where $l_t \in \mathbb{R}^{180}$ is the raw LiDAR range

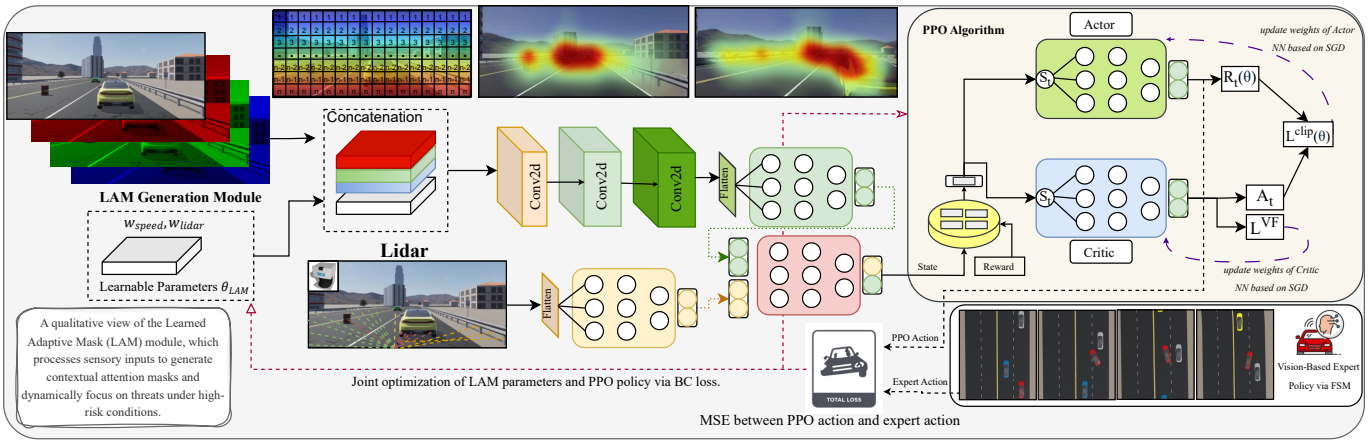

Fig. 2. Architecture of the LAM and its integration with the PPO policy. Normalized speed v_t^{norm} and hazard level h_t modulate a vertical gradient mask via learnable parameters α_{speed} and α_{lidar}. The resulting mask is concatenated with the RGB image to form a 4-channel input, enabling context-aware visual processing. LAM is trained end-to-end via BC gradients, allowing the agent to discover adaptive attention patterns that prioritize safety-critical regions.

vector, clipped at the maximum range $d_{\max}$. These signals are then scaled to the interval $[0, 1]$.

$$v_t^{\text{norm}} = \text{clamp}\left(\frac{v_t}{v_{\max}}, 0, 1\right) \tag{1}$$

$$h_t = \text{clamp}\left(\frac{d_{\text{safe}} - d_{\min,t}}{d_{\text{safe}}}, 0, 1\right) \tag{2}$$

where $v_{\max}$ and d_{safe} are predefined maximum speed and safety distance thresholds, respectively. LAM computes context-dependent modulation factors using two learnable scalar parameters $\alpha_{\text{speed}}, \alpha_{\text{lidar}} \in \mathbb{R}$. They are initialized to 0.5:

$$f_{\text{speed}} = 1 + \alpha_{\text{speed}} \cdot v_t^{\text{norm}}, \quad f_{\text{hazard}} = 1 + \alpha_{\text{lidar}} \cdot h_t \tag{3}$$

These factors scale a base lower-bound intensity weight $w_{\text{base, lower}} = 1.0$, while the upper-bound weight is fixed at $w_{\text{base, upper}} = 0.0$. The resulting dynamic lower intensity is:

$$w_{\text{lower}} = w_{\text{base, lower}} \cdot f_{\text{speed}} \cdot f_{\text{hazard}} \tag{4}$$

A smooth vertical gradient mask is then constructed for each row $y \in [0, H-1]$ of the input image:

$$M_t(y) = w_{\text{base, upper}} + (w_{\text{lower}} - w_{\text{base, upper}}) \cdot \frac{y}{H-1} \tag{5}$$

To ensure numerical stability and bounded output, the mask is normalized:

$$\hat{M}_t = \frac{M_t}{\max(M_t) + \varepsilon}, \quad \varepsilon = 10^{-6} \tag{6}$$

yielding $\hat{M}_t \in \mathbb{R}^{1 \times H \times W \times 1}$.

The normalized RGB image $I_t \in \mathbb{R}^{H \times W \times 3}$ (scaled to $[0, 1]$) is concatenated channel-wise with $\hat{M}_t$ to form a 4-channel input tensor:

$$I_t' = \text{concat}(I_t/255.0, \hat{M}_t) \in \mathbb{R}^{H \times W \times 4} \tag{7}$$

This augmented input is processed by a shared convolutional backbone, which also fuses embedded LiDAR features before feeding into the actor and critic heads.

The LAM parameters $\theta_{\text{LAM}} = \{\alpha_{\text{speed}}, \alpha_{\text{lidar}}\}$ are optimized end-to-end during BC alongside the policy using the Adam optimizer with learning rate η_{BC} and L2 regularization applied selectively to policy weights:

$$\mathcal{L}_{\text{BC}} = \frac{1}{B} \sum_{i=1}^{B} \left\| \pi_\theta(s_t^{(i)}) - a_{\text{expert}}^{(i)} \right\|^2 + \lambda_{\text{L2}} \sum_{p \in \theta_{\text{policy}}} \|p\|^2 \tag{8}$$

where gradients flow through the 4-channel observation to update θ_{LAM}. Training includes gradient clipping (max norm $G_{\max}$) and learning rate scheduling via plateau detection.

As shown in Figure 2, the learned masks adapt dynamically to the driving context. Rather than shifting the geometric center of attention, the mechanism modulates the intensity of the spatial gradient. At high speeds, the mask amplitude significantly increases (via f_{speed}), which strengthens the feature extraction across the entire driveable area, effectively expanding the usable visual range while maintaining a strong prior on the immediate lane path. Similarly, when proximity to obstacles is detected ($d_{\min,t}$), the hazard factor (f_{hazard}) further amplifies the mask intensity. This ensures that the network receives a sharper, high-contrast signal of the immediate surroundings for precise collision avoidance, boosting safety without relying on hand-crafted heuristic rules.

B. Multi-Phase Learning Curriculum

Policy learning happens in two main phases. First, BC gives the agent a stable and safe starting point by training the policy network on expert demonstrations. This warm start reduces risky, high-variance exploration and ensures the agent begins with reasonable behavior. Then, the policy is improved with PPO [29], which allows controlled on-policy exploration and lets the agent go beyond simple imitation.

To keep PPO updates aligned with expert behavior, an adversarial IRL module based on GAIL [30] is added. This module provides dense, behavior-focused reward signals. The hybrid reward is defined as:

$$r_t = w_{\text{env}} r_{\text{env}}(s_t, a_t) + w_{\text{irl}} r_{\text{irl}}(s_t, a_t) \tag{9}$$

where $r_{\text{env}}(s_t, a_t)$ denotes the environment reward and $w_{\text{env}} + w_{\text{irl}} = 1$. The IRL reward is:

$$r_{\text{irl}}(s_t, a_t) = -\log(1 - D_\psi(s_t, a_t) + \varepsilon) \tag{10}$$

it is bounded within a predefined range.

The curriculum begins with an FSM-partitioned expert dataset D_{expert}, enabling balanced sampling across driving modes. During the imitation phase, both policy and diffusion planner are trained via BC with interval $T_{\text{BC}}^{\text{init}}$. In the mixed phase, PPO updates use hybrid rewards, with discriminator trained every T_{disc} steps and BC continued adaptively every $T_{\text{BC}}^{\text{mixed}}$ steps. Safety interventions, which are triggered when $d_{\min,t} < d_{\text{safe}}$ or $|d_{\text{lane},t}| > e_{\text{lane}}$, are corrected and stored with a metadata flag indicating diffusion-based intervention. This progressive design, which moves from imitation to hybrid RL and then to safety distillation, leads to robust, safe, and efficient learning.

C. Expert Data Generation via FSM-Aware Experience Replay

A key part of the framework is a reliable and high-quality source of expert demonstrations. Instead of dealing with the noise and inconsistency of real human driving logs, an expert policy is built from scratch using a deterministic FSM policy π^*. As shown in Figure 3, this FSM controller moves smoothly between four behavior modes: Lane Following, Obstacle Avoidance, Driving Straight, and Returning, with transitions defined by simple sensor-based rules that describe the current driving situation. What sets this approach apart is the FSM-aware experience replay strategy. Each collected transition $(o_t, a_t^*, s_{\text{FSM},t})$ is stored in a separate buffer D_s that corresponds to its active FSM state s. The full expert dataset is then formed as follows:

$$D_{\text{expert}} = \bigcup_s D_s$$

This clear, state-based structure helps solve a common problem in autonomous driving datasets, where rare but important events are often underrepresented, such as passing through a narrow gap or recovering from a strong lane drift. By sampling mini-batches evenly across FSM states, the training process sees a balanced mix of normal cruising and challenging edge cases, instead of being dominated by simple highway driving.

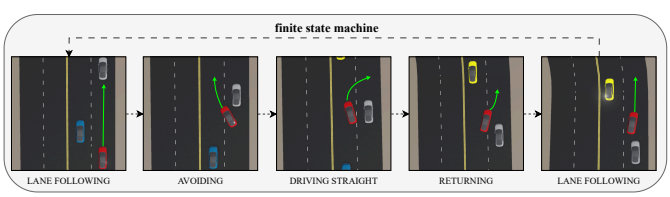

Fig. 3. The FSM expert policy in action. It switches smoothly between the modes (Lane Following, Obstacle Avoidance, Driving Straight, Returning) using sensor-based transition rules. The FSM-aware experience replay stores each sample in its matching state buffer, which gives balanced exposure to both normal and risky driving situations.

1) Phase 1: Foundational Pre-training: The training process starts with a supervised warm up over $N_{\text{imitation}}$ timesteps, using only D_{expert} to build a safe and reliable behavioral base. This phase protects the agent from unstable and unsafe exploration that often appears at the start of RL. Under BC, the policy π_θ is trained on compact embeddings $s_t = \phi(o_t)$ from the perception module. It learns to match the expert actions using a mean-squared error loss. Balanced sampling from the FSM-partitioned buffers ensures that all driving modes are well represented during training:

$$\mathcal{L}_{\text{BC}}(\theta) = \mathbb{E}_{(o_t, a_t^*) \sim D_{\text{expert}}^{\text{balanced}}} \left[\|\pi_\theta(s_t) - a_t^*\|^2 \right] + \lambda_{\text{L2}} \sum_{p \in \theta_{\text{policy}}} \|p\|^2 \tag{11}$$

This loss keeps the policy close to expert behavior long before any reward signal is used. Diffusion planner training runs concurrently with BC. A conditional 1D U-Net diffusion planner is trained to generate smooth and feasible motion sequences. It is trained on consecutive expert action chunks $\{a_t^*, \ldots, a_{t+H-1}^*\}$ taken from D_{expert} and optimized with the standard DDPM denoising objective [31]. In this way, the planner learns the smooth and physically consistent control patterns of the FSM expert and is prepared to act later as a reliable safety net.

2) Phase 2: Online Fine-tuning with Adversarial Reinforcement Learning: After a strong base is built, the system runs for N_{mixed} timesteps of online refinement with PPO. During this phase, the policy is improved through interaction with the environment, using a hybrid reward that combines clear task feedback with imitation-based signals. Under GAIL, a discriminator D_ψ is trained to go beyond simple action matching. It learns to tell the difference between policy rollouts (s_t, a_t) and true expert pairs (s_t^*, a_t^*) using a binary cross-entropy loss:

$$\begin{aligned} \mathcal{L}_{\text{Disc}}(\psi) = &-\mathbb{E}_{(s_t, a_t) \sim \pi_\theta}[\log(1 - D_\psi(s_t, a_t))] \\ &-\mathbb{E}_{(s_t^*, a_t^*) \sim D_{\text{expert}}}[\log(D_\psi(s_t^*, a_t^*))] \end{aligned} \tag{12}$$

Once training is complete, the discriminator provides a dense and continuous imitation reward:

$$r_{\text{irl}}(s_t, a_t) = -\log(1 - D_\psi(s_t, a_t) + \varepsilon) \tag{13}$$

Balanced sampling across FSM states ensures that the discriminator sees the full range of expert behaviors.

In the hybrid reward formulation, the final PPO reward is designed to balance task completion with expert-level behavior.

$$r_t(s_t, a_t) = w_{\text{env}} r_{\text{env}}(s_t, a_t) + w_{\text{irl}} r_{\text{irl}}(s_t, a_t) \tag{14}$$

where $w_{\text{env}} + w_{\text{irl}} = 1$. This combination helps the agent achieve high-level goals, such as steady progress and zero collisions, while also matching the smooth, anticipatory style of the FSM expert.

D. Safety Assurance via Guided Diffusion and Experience Correction

Finally, a diffusion-based planner is added as a safety-focused backup system. The DAL module is activated only

in high-risk situations and quickly generates short, feasible paths that keep the vehicle safe. At the same time, it improves learning by storing safe, corrected experiences in the policy memory, so the agent becomes better over time without reinforcing unsafe behaviors. Together, these parts (perception, curriculum, expert replay, and safety) form an integrated pipeline that combines fast reactions with careful planning and leads to behavior that is effective, safe, and close to human driving.

Even when the PPO policy learns complex driving patterns, it is still hard to stay safe in rare or unexpected situations. To reduce these risks, the framework uses two safety components:

1) An on-demand, energy-guided diffusion planner that is activated to generate safe actions when the situation becomes dangerous.
2) An experience correction mechanism that sends these safe corrections back to the main policy so that temporary fixes become lasting improvements.

1) On-Demand, Energy-Guided Trajectory Generation: The diffusion planner trained in the first phase now acts as a safety-critical motion generator. It stays inactive most of the time to save computation and is only turned on in high-risk states. A state is treated as high-risk when the closest LiDAR reading becomes too small or the vehicle moves far away from the lane center:

$$d_{\min,t} < d_{\text{trigger}} \quad \vee \quad |d_{\text{lane},t}| > e_{\text{lane}} \tag{15}$$

letting the PPO policy drive freely under normal conditions.

When DAL is called, it generates a short safe trajectory $\mathcal{A} = a_t, \ldots, a_{t+H-1}$ based on the current context embedding. This is done with a guided reverse diffusion process. At each denoising step k, the U-Net predicts a cleaner trajectory $\hat{\mathcal{A}}_k^0$, which is then moved toward safer behavior by following the gradient of a composite energy function $E(\mathcal{A}, o_t)$:

$$\tilde{\mathcal{A}}_k^0 = \hat{\mathcal{A}}_k^0 - w_g \cdot \frac{\nabla E(\hat{\mathcal{A}}_k^0, o_t)}{\|\nabla E\| + \varepsilon} \tag{16}$$

where w_g determines how strongly the planner pushes trajectories toward safe regions during sampling, and $\tilde{\mathcal{A}}_k^0$ denotes the resulting safety-guided trajectory obtained after applying the energy gradient.

The energy E is a weighted sum of five simple terms that together enforce lane keeping, obstacle clearance, smooth control, stability, and an optional expert-alignment term.

$$E = E_{\text{lane}} + E_{\text{lidar}} + E_{\text{jerk}} + E_{\text{stability}} + E_{\text{expert}} \tag{17}$$

Lane Adherence — E_{lane}: This term penalizes deviation from the lane center. Instead of computationally expensive forward modeling, the energy term utilizes the current lateral error state distance (d_{lat}) to strictly guide the diffusion sampling process toward immediate correction.

$$E_{\text{lane}} = w_{\text{lane}} \left(\frac{d_{\text{lane},t}}{s_{\text{lane}}} \right)^2 \tag{18}$$

with a risk-adaptive weight w_{lane} that increases under higher hazard levels:

$$w_{\text{lane}} = w_{\text{base}}^{\text{lane}} (1 + \alpha_{\text{hazard}} h_t)$$

where $w_{\text{base}}^{\text{lane}}$ is the nominal weighting coefficient for lane keeping, α_{hazard} controls how aggressively the weight increases under hazardous conditions, and h_t denotes the instantaneous hazard indicator.

Obstacle Avoidance — E_{lidar}: To keep a safe distance from obstacles, trajectories generated while the vehicle is in close proximity to obstacles are penalized based on the current sensor state ($d_{\min,t}$).

$$E_{\text{lidar}} = w_{\text{lidar}} \cdot \max \left(0, \frac{d_{\text{safe}}^{\text{plan}} - d_{\min,t}}{s_{\text{lidar}}} \right)^2 \tag{19}$$

where the weight w_{lidar} increases with the hazard level, according to $w_{\text{lidar}} = w_{\text{base}}^{\text{lidar}}(1 + h_t)$.

Control Smoothness — E_{jerk}: To avoid jerky or hard-to-drive behavior, large changes between consecutive actions are penalized:

$$E_{\text{jerk}} = w_{\text{jerk}} \cdot \frac{1}{H-1} \sum_{i=1}^{H-1} \|a_i - a_{i-1}\|^2 \tag{20}$$

where w_{jerk} is the weighting coefficient for smoothness and H is the trajectory horizon.

Stability — $E_{\text{stability}}$: Steady, centered control is encouraged by penalizing large steering or speed deviations from a neutral reference:

$$E_{\text{stability}} = w_{\text{stab}} \cdot \frac{1}{H} \sum_{i=0}^{H-1} \left(\text{steer}_i^2 + (\text{speed}_i - v_{\text{ref}})^2 \right) \tag{21}$$

where w_{stab} is the stability weight and v_{ref} denotes the target reference speed.

Expert Alignment — E_{expert} (optional): When an expert reference is available, an additional term keeps the trajectory close to the expert demonstration:

$$E_{\text{expert}} = w_{\text{exp}} \cdot \frac{1}{H} \sum_{i=0}^{H-1} \|a_i - a_i^{\text{expert}}\|^2 \tag{22}$$

where w_{exp} controls the strength of expert-matching and a_i^{expert} is the corresponding expert action.

The hazard level h_t adapts all risk-sensitive weights:

$$h_t = \text{clamp}\left(1 - \tanh\left(\frac{d_{\min,t}}{s_h} \right), 0, 1 \right) \tag{23}$$

where $d_{\min,t}$ is the minimum LiDAR distance at time t, s_h is a scaling factor determining sensitivity to obstacles, and the clamp limits the value to the range $[0, 1]$.

Conditioning is performed using a compact context vector $c_t \in \mathbb{R}^{64}$, which encodes current hazard level, lateral deviation, speed, steering, and LiDAR statistics. Each element is normalized, and unused entries are padded for dimensional consistency.

2) Action Blending and Execution: Only the first action a_t^{DAL} from the refined trajectory is used. During the mixed phase, this action is smoothly blended with the PPO action using a dynamic blend weight:

$$w_b = \begin{cases} 1.0, & d_{\min,t} < d_{\text{critical}} \\ w_{\text{base}}^{\text{blend}} + w_{\text{scale}}^{\text{blend}} h_{\text{blend}} & \text{otherwise} \end{cases} \tag{24}$$

where:

$$h_{\text{blend}} = e^{-d_{\min,t}/s_1} + k_{\tanh} \tanh(|d_{\text{lane},t}|/s_2) \tag{25}$$

and w_b is temporally smoothed via EMA. The final executed action is:

$$a_t^{\text{final}} = w_b a_t^{\text{DAL}} + (1 - w_b) a_t^{\text{PPO}} \tag{26}$$

3) Experience Correction: The executed transition is stored in the PPO replay buffer with a correction marker, and any unsafe PPO action is replaced by the safe blended output to prevent accidents (Runtime Shielding). However, the experience stored in the replay buffer remains anchored to the FSM expert action. This ensures the policy learns stable rule-based behaviors while the diffusion layer prevents premature episode termination, as illustrated in Fig. 4.

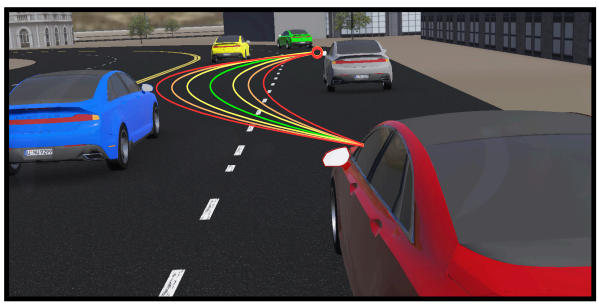

Fig. 4. The safety pipeline in action. DAL is activated by high-risk signals, generates energy-guided safe trajectories, and blends the first action with the PPO output. The corrected experiences are stored in the replay buffer with markers, The diffusion planner acts as an active shield, allowing the agent to safely collect more high-quality expert data even in challenging scenarios.

E. Safety-Aware Experience Correction (SAEC)

To ensure robust training without premature episode termination, the SAEC mechanism employs the diffusion planner as a runtime safety shield. As described in Algorithm 1, the planner is activated when the agent enters a high-risk state, defined by:

$$d_{\min,t} < d_{\text{trigger}} \vee |d_{\text{lane},t}| > e_{\text{lane}} \tag{27}$$

where $d_{\min,t}$ is the closest LiDAR reading and $d_{\text{lane},t}$ measures the lateral deviation. Once triggered, DAL generates a safe action to prevent immediate collision. However, to ensure the policy converges to a stable kinematic behavior, we do not train on the corrective diffusion action itself. Instead, we utilize the concurrent Rule-Based Expert (FSM) to calculate the ideal ground-truth action (a_{Expert}) for that specific state.

The transition stored in the PPO replay buffer D_{PPO} is therefore defined as:

$$D_{\text{PPO}} \leftarrow D_{\text{PPO}} \cup (o_t, a_{\text{Expert}}, r_t, o_{t+1}) \tag{28}$$

This mechanism provides two critical benefits:

- **Runtime Survival:** The diffusion action is executed to physically prevent collisions, allowing the episode to continue beyond states that would normally cause termination.
- **Safe Expert Labeling:** By keeping the agent active in near-accident scenarios, the system allows the FSM expert to label these "edge cases" with correct recovery actions. The policy π_θ thus learns obstacle avoidance based on the FSM's consistent logic, while the diffusion layer acts solely as a guardrail during this learning process.

F. Policy Optimization with Safety-Aware Curriculum

The core of the system, the driving policy, is trained with a safety-first curriculum that combines the stable reliability of IL with the flexibility of RL. This combination leads to behavior that is not only skilled but also naturally cautious and robust. The full training procedure is summarized in Algorithm 1.

1) RL Backbone: PPO: To improve the policy through interaction with the environment, PPO is used, an on-policy actor-critic method well suited to smooth, high-dimensional control. Its clipped objective keeps updates stable and prevents large policy changes that could disrupt training. The full optimization objective is:

$$L(\theta) = \hat{\mathbb{E}}_t \left[L_t^{\text{CLIP}}(\theta) + c_1 L_t^{\text{VF}}(\phi) - c_2 S[\pi_\theta](s_t) \right] \tag{27}$$

where L_t^{CLIP} is the clipped policy loss, L_t^{VF} is the value function error, and $S[\pi_\theta]$ is an entropy term that encourages the policy to keep exploring. Raw observations are first converted into compact embeddings $s_t = \phi(o_t)$ by a shared perception backbone (Sec. III-A). After this step, the actor and critic each use their own lightweight MLP head.

2) Multi-Phase Training Curriculum: Training is carried out in two phases. It starts with an imitation warm up and then moves to a reward driven refinement phase.

a) Phase 1 — Imitation Pre-training:: Over the first $N_{\text{imitation}}$ timesteps, both the policy π_θ and diffusion planner are bootstrapped via BC on the FSM-aware expert dataset D_{expert} (Sec. III-C). Balanced sampling across driving modes guarantees exposure to rare but critical edge cases:

$$\mathcal{L}_{\text{BC}} = \mathbb{E}_{(o_t, a_t^*) \sim D_{\text{expert}}^{\text{balanced}}} \left[\|\pi_\theta(s_t) - a_t^*\|^2 \right] + \lambda_{\text{L2}} \|\theta_{\text{policy}}\|^2. \tag{28}$$

This phase plants a **safe, expert-grade behavioral prior**, slashing unsafe exploration right from the start.

b) Phase 2 — IRL-PPO Fine-tuning with Hybrid Reward:: Once a solid foundation is set, the agent goes for N_{mixed} timesteps, refining itself with PPO under a **hybrid reward** that fuses crisp environmental feedback with rich, learned imitation cues:

$$r_t = w_{\text{env}} r_{\text{env}}(s_t, a_t) + w_{\text{irl}} r_{\text{irl}}(s_t, a_t) \tag{29}$$

where $w_{\text{env}} + w_{\text{irl}} = 1$.

IRL Reward from GAIL discriminator:

$$r_{\text{irl}}(s_t, a_t) = -\log(1 - D_\psi(s_t, a_t) + \varepsilon), \quad r_{\text{irl}} \in [r_{\min}, r_{\max}]$$

The environment reward r_{env} is dense and FSM aware. It is adapted to the current driving mode and combines terms for precise lane centring, obstacle clearance, and sparse goal completion. A collision gives a large negative reward, a near miss gives a smaller penalty, and a minimum baseline keeps the reward positive in safe states, as illustrated in Fig. 5.

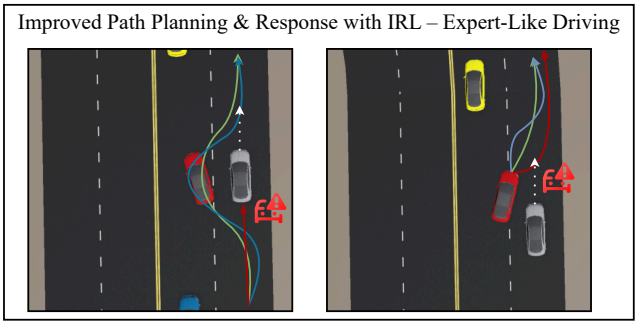

Fig. 5. Impact of Hybrid Reward Shaping on Trajectory Smoothness. The blue trajectory illustrates the policy trained solely on the rule-based environment reward (r_{env}), resulting in oscillatory behavior and excessive lateral deviation. In contrast, the green trajectory incorporates the dense IRL signal (r_{IRL}), effectively regularizing the policy towards the expert's kinematic profile. This results in smoother lane-change maneuvers (left) and tighter collision avoidance (right), correcting the overshooting tendencies observed in the baseline.

G. Integrated Training Pipeline

Algorithm 1 ties everything together: FSM-aware replay, BC, GAIL, PPO, DAL, and SAEC.

Algorithm 1: Hybrid IRL–DAL Training Framework

Initialize: π_θ, V_ϕ, P_{DAL}, D_ψ, $\mathcal{D}_{\text{buffer}} \leftarrow \emptyset$
Phase 1: Imitation (Warm-up & Data Collection)
for $t \leftarrow 0$ **to** $N_{imitation} - 1$ **do**
  Observe o_t, compute expert action $a_t^* \leftarrow \text{FSM}(o_t)$
  Execute a_t^*, observe o_{t+1}
  Store (o_t, a_t^*) in $\mathcal{D}_{\text{buffer}}$
  if $t \bmod T_{BC} = 0$ **then**
    Update π_θ via BC on $\mathcal{D}_{\text{buffer}}$ (Eq. 28)
  end
  if $t \bmod T_{diffusion} = 0$ **then**
    TRAINDIFFUSION($\mathcal{D}_{\text{buffer}}$)
  end
end
Phase 2: Mixed (RL + DAL Safety + Regularization)
for $t \leftarrow N_{imitation}$ **to** $N_{total} - 1$ **do**
  Sample agent action $a_t^{\text{PPO}} \sim \pi_\theta(o_t)$
  Compute expert action $a_t^* \leftarrow \text{FSM}(o_t)$
  if $d_{\min,t} < d_{trigger}$ ***or*** $|d_{lane,t}| > e_{lane}$ **then**
    Sample $\mathcal{A} \sim P_{\text{DAL}}(o_t)$ with energy guidance
    $a_t^{\text{final}} \leftarrow \text{blend}(a_t^{\text{DAL}}, a_t^{\text{PPO}})$
  else
    $a_t^{\text{final}} \leftarrow a_t^{\text{PPO}}$
  end
  Execute a_t^{final}, observe r_{env}, o_{t+1}
  $r_{\text{irl}} \leftarrow -\log(1 - D_\psi(o_t, a_t^{\text{final}}) + \varepsilon)$
  $r_t \leftarrow w_{\text{env}} r_{\text{env}} + w_{\text{irl}} r_{\text{irl}}$
  Store $(o_t, a_t^{\text{final}}, a_t^*, r_t)$ in $\mathcal{D}_{\text{buffer}}$
  if $t \bmod T_{disc} = 0$ **then**
    TRAINDISCRIMINATOR($\mathcal{D}_{\text{buffer}}$)
  end
  if $t \bmod T_{sync} = 0$ **then**
    SYNCFEATURES()
  end
  if SHOULD_RUN_BC_TRAINING() **then**
    TRAINBC($\mathcal{D}_{\text{buffer}}$)
  end
  Update π_θ, V_ϕ via PPO using collected rollouts
end

IV. Simulations and Results

A. Experimental Setup

All experiments are done in the Webots simulator. Webots provides realistic vehicle motion and flexible scene settings. The virtual city has multi-lane curved roads, moving obstacles, and different lighting conditions from bright day to dark evening. This setup makes the tests close to real urban driving. For reproducibility, all details of the perception backbone are given. It takes a dictionary-style observation that includes camera images and LiDAR scans.

1) **Input Data:** The model takes two input streams. The visual stream is a 4-channel tensor $I'_t \in \mathbb{R}^{H \times W \times 4}$, formed from a downscaled RGB image and the LAM output. The auxiliary stream is a 1D LiDAR scan $L_t \in \mathbb{R}^{N_{\text{beams}}}$ that provides distance measurements in all directions.
2) **Visual Encoder:** A three-layer convolutional network with kernel sizes 5, 3, and 3 processes the visual input. It uses ReLU activations and stride 2 for downsampling, and a final flatten layer produces a compact visual feature vector z_{vision}.
3) **LiDAR Encoder:** The LiDAR vector is first normalized and then passed through a three-layer MLP with ReLU activations. This network produces a 32-dimensional embedding z_{lidar} that captures the scene geometry.
4) **Feature Fusion:** The two latents are concatenated and polished by a small fusion MLP to produce the final state $s_t \in \mathbb{R}^{512}$:

$$\boldsymbol{s}_t = \text{MLP}_{\text{fusion}}(\text{concat}[\boldsymbol{z}_{\text{vision}}, \boldsymbol{z}_{\text{lidar}}]). \tag{30}$$

This fused vector gives the policy a complete view of the scene, with fine local structure from LiDAR and global context from vision, and it is used in both the imitation warm-up and the later online RL training.
5) **Environment and Episode Termination:** A custom Gym wrapper updates observations at 10 Hz. At each step, it provides an $H \times W \times 4$ visual tensor (RGB plus mask) and an N_{beams}-beam LiDAR scan. An episode ends either when a collision occurs ($d_{\min} < d_{\text{collision}}$) or when the goal is reached. A successful episode is one that reaches the goal without any violations.
6) **Training Procedure:** Training followed the two-phase safety-aware curriculum shown in Algorithm 1, for a total of N_{total} timesteps. All transitions were stored in FSM-partitioned replay buffer [33], so that rare safety-related events are well represented. The main hyperparameters are given in Table II.

B. Component-wise Evaluation of the IRL–DAL Framework

To measure the effect of each component, a clear ablation study was carried out. Four model variants were trained

TABLE I
QUANTITATIVE PERFORMANCE ACROSS ARCHITECTURAL VARIANTS (10 SEEDS, MEAN ± STD). MEAN REWARD NORMALIZED TO [0, 200]. TRAJECTORY PREDICTION METRICS (ADE/FDE) FROM ROLLOUT EVALUATION. ARROWS INDICATE IMPROVEMENT DIRECTION; **BOLD** DENOTES BEST.

Model	Mean Reward ↑	Coll./1k Steps ↓	Success (%) ↑	BC Loss ($\times 10^{-2}$) ↓	Action Sim. (%) ↑	ADE (m) ↓	FDE (m) ↓
PPO + Uniform Samplingc[32]	85.2 ± 4.1	0.63 ± 0.12	78.1 ± 3.2	17.1 ± 1.4	65.3 ± 4.1	5.25 ± 0.31	11.8 ± 0.65
+ FSM Replay[33]	120.4 ± 3.8 (+41%)	0.30 ± 0.08	88.4 ± 2.1	12.3 ± 1.1	75.1 ± 3.5	4.10 ± 0.27	9.5 ± 0.58
+ Diffusion Planner[34]	155.1 ± 3.2 (+29%)	0.15 ± 0.05	92.0 ± 1.8	13.0 ± 1.0	80.2 ± 3.0	3.15 ± 0.22	7.2 ± 0.49
IRL–DAL (Ours)	**180.7 ± 2.9** (+16%)	**0.05 ± 0.03**	**96.3 ± 1.2**	**7.4 ± 0.8**	**85.7 ± 2.4**	**2.45 ± 0.18**	**5.1 ± 0.41**

TABLE II
TRAINING HYPERPARAMETERS

Parameter	Value (from code)
PPO Steps / Batch / Epochs	2048 / 64 / 10
Discount Factor (γ) / GAE-λ	0.99 / 0.95
BC / PPO Learning Rate	3×10^{-4}
Diffusion Horizon (H) / Diffusion Steps (T)	8 / 100
Total Buffer Capacity	50,000 transitions
IRL Reward Weight (w_{irl})	0.3
Environment Reward Weight (w_{env})	0.7
Imitation Phase Duration ($N_{\text{imitation}}$)	20,000 steps
Mixed Phase Duration (N_{mixed})	30,000 steps
Total Training Steps (N_{total})	50,000 steps
BC Update Interval (Imitation)	every 1,000 steps
BC Update Interval (Mixed)	every 500 steps
Diffusion Training Interval	every 500 steps
Discriminator Update Interval (T_{disc})	every 2,000 steps
Feature Sync Interval (T_{sync})	every 1,000 steps
L2 Regularization (λ_{L2})	10^{-6}
PPO Clip Range	0.2
Entropy Coefficient (c_2)	0.01
Value Loss Coefficient (c_1)	0.5
Gradient Clip Norm ($G_{\max}$)	0.5
Learning Rate Scheduler	Reduce on Plateau (patience=5)
LiDAR Beams (N_{beams})	180
Image Size ($H \times W$)	64×64
Collision Threshold ($d_{\text{collision}}$)	1.0 m
DAL Trigger: d_{trigger} / e_{lane}	3.0 m / 120 px
Critical Blend Threshold (d_{critical})	1.5 m
Hazard Scale (s_h)	3.0
Blend Hazard Scales (s_1, s_2, k_{tanh})	2.0 / 20 / 0.3
DAL Guidance Weight (w_g)	0.1
Energy Weights (w_{jerk}, w_{stab}, w_{exp})	0.1 / 0.5 / 2.0
Lane Scale (s_{lane}) / LiDAR Scale (s_{lidar})	0.5 / 2.0
LAM Alphas Initial Value	0.5
Optimizer	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)

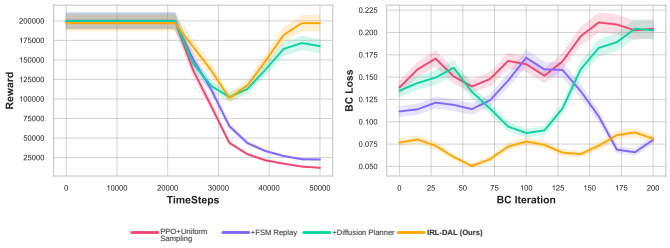

Fig. 6. Training dynamics over 50k steps. After switching to mixed-mode at 20k steps, the full IRL–DAL model (*Diff+Adapt+SAEC*) rockets ahead in reward and stability. BC loss stays pinned low, and DAL interventions fade to near-zero—proof of **adaptive distillation**. See Table I for final numbers.

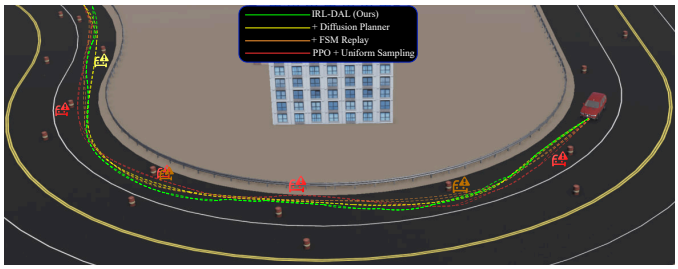

Fig. 7. Qualitative ablation study on a high-curvature turn scenario in Webots. The **Baseline PPO (Red)** fails to negotiate the curve, resulting in a collision. Adding **FSM Replay (Orange)** improves lane adherence but exhibits oscillatory behavior. The **Diffusion Planner (Yellow)** successfully ensures safety but deviates from the center. The full **IRL–DAL framework (Green)** demonstrates superior trajectory smoothness and precise lane centering, attributing to the LAM's focused attention on road boundaries.

under the same conditions, adding components one by one to separate their individual contributions. The evaluated variants are as follows: The Baseline (PPO + Uniform Sampling) is the standard PPO algorithm [32] with a uniform replay buffer. The second variant, + FSM Replay, builds upon the baseline model by incorporating FSM-aware replay, which aims to provide more coverage to safety-critical states. The third variant, + Diffusion Planner, includes the structured replay along with the diffusion-based safety planner [34], but without adaptive perception. Finally, the + LAM + SAEC (Full IRL-DAL) represents the full model, incorporating the adaptive mask, safety-aware experience correction, and all components enabled. These results are summarized in Table I and visualized in Fig. 6, which display reward curves, BC alignment, and intervention decay.

Structured replay and safety awareness have a key role in the results. Using only the FSM buffer increases the reward by 41% and reduces collisions by 52% (from 0.63 to 0.30 per 1k steps), which shows that rare events need special attention. Adding DAL increases the reward by another 29% and again cuts collisions by half (from 0.30 to 0.15), so it not only reacts to danger but also helps prevent it (Fig. 7). With the full model, including adaptive perception (LAM) and SAEC, collisions drop to 0.05, which is a 67% reduction compared to the previous step and 12.6 times better than the baseline. At the same time, BC loss stays low and action similarity to the expert increases, so expert-like behavior is reached without sacrificing stability.

V. CONCLUSION

This work introduced IRL-DAL, a unified framework for safe and adaptive autonomous driving that combines inverse reinforcement learning, diffusion planning, and on-policy control. The method links three key ideas in a single loop: hybrid IL-IRL-RL training, a diffusion-based safety planner, and a LAM for perception. Together, these parts let the agent learn an expert-like driving policy that remains stable under online interaction and reacts safely in high-risk situations.

The framework was evaluated in the Webots simulator using a two-phase curriculum. In the first phase, an FSM expert and behavioral cloning provided a safe and reliable starting point for both the policy and the diffusion planner. In the second phase, PPO with a hybrid reward and GAIL-based IRL signals refined the behavior while DAL and SAEC enforced safety

and turned interventions into useful training data. The final agent reached a 96% success rate and reduced collisions to 0.05 per 1k steps. Ablation studies showed clear gains from each component: FSM-aware replay improved coverage of rare events, the diffusion planner reduced further failures, and LAM plus SAEC delivered the larger safety and performance gains.

REFERENCES

[1] L. Crosato, K. Tian, H. P. Shum, E. S. Ho, Y. Wang, and C. Wei, "Social interaction-aware dynamical models and decision-making for autonomous vehicles," *Advanced Intelligent Systems*, vol. 6, no. 3, p. 2300575, 2024.

[2] B. R. Kiran, I. Sobh, V. Talpaert, P. Mannion, A. E. Sallab, S. Yogamani, and P. Pérez, "Deep reinforcement learning for autonomous driving: A survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 6, pp. 4909–4926, 2021.

[3] S. Ross, G. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, 2011, pp. 627–635.

[4] M. Cui, Y. Hu, S. Xu, J. Wang, Z. Bing, B. Li, and A. Knoll, "Safe and human-like trajectory planning of self-driving cars: A constraint imitative method," *Advanced Intelligent Systems*, vol. 5, no. 10, p. 2300269, 2023.

[5] G. Li, Z. Ji, S. Li, X. Luo, and X. Qu, "Driver behavioral cloning for route following in autonomous vehicles using task knowledge distillation," *IEEE Transactions on Intelligent Vehicles*, vol. 8, no. 2, pp. 1025–1033, 2022.

[6] X. Wang, C. Wei, H. Tian, W. Wang, and J. Hu, "Implicit predictive behavior cloning for autonomous driving decision-making in urban traffic," *IEEE Transactions on Intelligent Vehicles*, 2024.

[7] H. M. Eraqi, M. N. Moustafa, and J. Honer, "Dynamic conditional imitation learning for autonomous driving," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 12, pp. 22 988–23 001, 2022.

[8] C. Yuan, C. Shuai, Z. Zhang, J. Ma, Y. Fang, and Y. Sun, "Model-free deep reinforcement learning with multiple line-of-sight guidance laws for autonomous underwater vehicles full-attitude and velocity control," *Advanced Intelligent Systems*, p. 2400991, 2025.

[9] M. O. Radwan, A. A. H. Sedky, and K. M. Mahar, "Obstacles avoidance of self-driving vehicle using deep reinforcement learning," in *2021 31st International Conference on Computer Theory and Applications (ICCTA)*. IEEE, 2021, pp. 215–222.

[10] R. Mahmoudi and A. Ostreika, "Reinforcement learning for obstacle avoidance application in unity ml-agents." in *IVUS*, 2023, pp. 214–221.

[11] F. Pinto and S. Al-Stouhi, "Curriculum reinforcement learning for autonomous driving," in *SAE World Congress Experience*. SAE International, 2021.

[12] Y. Lu, J. Fu, G. Tucker, X. Pan, E. Bronstein, R. Roelofs, B. Sapp, B. White, A. Faust, S. Whiteson *et al.*, "Imitation is not enough: Robustifying imitation with reinforcement learning for challenging driving scenarios," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 7553–7560.

[13] R. Zhao, Y. Li, Y. Fan, F. Gao, M. Tsukada, and Z. Gao, "A survey on recent advancements in autonomous driving using deep reinforcement learning: Applications, challenges, and solutions," *IEEE Transactions on Intelligent Transportation Systems*, 2024.

[14] G. Lanzaro and T. Sayed, "Evaluating driver-pedestrian interaction behavior in different environments via markov-game-based inverse reinforcement learning," *Expert Systems with Applications*, vol. 260, p. 125405, 2025.

[15] Z. Huang, H. Liu, J. Wu, and C. Lv, "Conditional predictive behavior planning with inverse reinforcement learning for human-like autonomous driving," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 7, pp. 7244–7258, 2023.

[16] P. Wang, D. Liu, J. Chen, H. Li, and C.-Y. Chan, "Decision making for autonomous driving via augmented adversarial inverse reinforcement learning," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 1036–1042.

[17] C. Lu, S. Ali, and T. Yue, "Epitester: Testing autonomous vehicles with epigenetic algorithm and attention mechanism," *IEEE Transactions on Software Engineering*, 2024.

[18] X. Xi, X. Jin, Q. Jiang, Y. Lin, W. Zhou, and L. Guo, "Ema-gan: A generative adversarial network for infrared and visible image fusion with multiscale attention network and expectation maximization algorithm," *Advanced Intelligent Systems*, vol. 5, no. 11, p. 2300310, 2023.

[19] Y. Wang, W. Wan, H. Zhang, C. Chen, and G. Jia, "Pedestrian trajectory intention prediction in autonomous driving scenarios based on spatio-temporal attention mechanism," in *2024 4th International Conference on Electronic Information Engineering and Computer Communication (EIECC)*. IEEE, 2024, pp. 1519–1522.

[20] M. Schwonberg, J. Niemeijer, J.-A. Termöhlen, N. M. Schmidt, H. Gottschalk, T. Fingscheidt *et al.*, "Survey on unsupervised domain adaptation for semantic segmentation for visual perception in automated driving," *IEEE Access*, vol. 11, pp. 54 296–54 336, 2023.

[21] S. A. H. Miangoleh, A. J. Aghdasian, and F. Abdollahi, "Blip-fuseppo: A vision-language deep reinforcement learning framework for lane keeping in autonomous vehicles," 2025. [Online]. Available: https://arxiv.org/abs/2510.22370

[22] X. Chi, Z. Guo, and F. Cheng, "Dynamic obstacle avoidance model of autonomous driving with attention mechanism and temporal residual block," *Alexandria Engineering Journal*, vol. 105, pp. 538–548, 2024.

[23] X. Ma, K. Hu, X. Sun, and S. Chen, "Adaptive attention module for image recognition systems in autonomous driving," *International Journal of Intelligent Systems*, vol. 2024, no. 1, p. 3934270, 2024.

[24] M. Peng, K. Chen, X. Guo, Q. Zhang, H. Zhong, M. Zhu, and H. Yang, "Diffusion models for intelligent transportation systems: A survey," *IEEE Transactions on Intelligent Transportation Systems*, 2025.

[25] J. Wang, J. Guo, M. Feng, C. Li, X. Xue, and J. Pu, "Trajectory grid diffusion for multimodal trajectory prediction in autonomous vehicles," *IEEE Transactions on Intelligent Vehicles*, 2024.

[26] T. Pearce, A. Canonaco, H. Zhu, I. Havoutis, and N. Pugeault, "Imitating human behaviour with diffusion models," *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3607–3614, 2023.

[27] C. Chi, V. Little, S. Zhang, Z. Wang, S. Wang, A. Chang, and M. Sung, "Diffusion-based generation, optimization, and planning in 3d scenes," in *Conference on Robot Learning*, 2023, pp. 2209–2220.

[28] Z. Wang, M. Janner, Y. Du, C. Finn, and S. Levine, "Diffusion policies as an expressive policy class for offline reinforcement learning," in *International Conference on Learning Representations*, 2023.

[29] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[30] J. Ho and S. Ermon, "Generative adversarial imitation learning," *Advances in neural information processing systems*, vol. 29, 2016.

[31] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.

[32] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017. [Online]. Available: https://arxiv.org/abs/1707.06347

[33] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "Carla: An open urban driving simulator," 2017. [Online]. Available: https://arxiv.org/abs/1711.03938

[34] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," 2024.